

HDOC Day-33

Write a C program to perform the following menu-driven operations using separate functions:

1. Read three variable-length 2D integer arrays A, B, and C, each of dimensions (m x n), and verify the associative property of matrix addition: $(A+B)+C=A+(B+C)$
2. Read three variable-length 2D integer arrays with the following dimensions: A (m x n), B (n x p), C (p x q). Verify the associative property of matrix multiplication: $(A \times B) \times C = A \times (B \times C)$
3. Exit

Algorithm

Operation 1: Matrix Addition Associativity

1. Start.
2. Display the menu.
3. Read the user's choice.
4. If choice = 1:
 - Read m and n.
 - Dynamically allocate matrices A, B, and C of size $m \times n$.
 - Read elements of all three matrices.
 - Calculate: $AB=A+B$
 - Calculate: $Left=(A+B)+C$
 - Calculate: $BC=B+C$
 - Calculate: $Right=A+(B+C)$
 - Display Left and Right.
 - Compare both matrices.
 - If they are equal, display **Associative property is VERIFIED**.
5. Free the allocated memory.

Operation 2: Matrix Multiplication Associativity

1. Read m, n, p, and q.

2. Allocate:
 - A as $m \times n$
 - B as $n \times p$
 - C as $p \times q$
3. Read all matrix elements.
4. Calculate: $AB = A \times B$
5. Calculate: $Left = (A \times B) \times C$
6. Calculate: $BC = B \times C$
7. Calculate: $Right = A \times (B \times C)$
8. Display both matrices.
9. Compare both matrices.
10. If equal, display **Associative property is VERIFIED**.
11. Free allocated memory.

● C Program:

- `#include <stdio.h>`
- `#include <stdlib.h>`
-
- `/* Function to read a matrix */`
- `void readMatrix(int **matrix, int rows, int cols)`
- `{`
- `int i, j;`
-
- `printf("Enter elements of the matrix:\n");`
-
- `for (i = 0; i < rows; i++)`
- `{`
- `for (j = 0; j < cols; j++)`
- `{`
- `scanf("%d", &matrix[i][j]);`
- `}`
- `}`
- `}`
-

- /* Function to display a matrix */
- void displayMatrix(int **matrix, int rows, int cols)
- {
- int i, j;
-
- for (i = 0; i < rows; i++)
- {
- for (j = 0; j < cols; j++)
- {
- printf("%d\t", matrix[i][j]);
- }
- printf("\n");
- }
- }
-
- /* Function to add two matrices */
- void addMatrix(int **A, int **B, int **result, int rows, int cols)
- {
- int i, j;
-
- for (i = 0; i < rows; i++)
- {
- for (j = 0; j < cols; j++)
- {
- result[i][j] = A[i][j] + B[i][j];
- }
- }
- }
-
- /* Function to multiply two matrices */
- void multiplyMatrix(int **A, int **B, int **result,
- int r1, int c1, int c2)
- {
- int i, j, k;
-
- for (i = 0; i < r1; i++)
- {
- for (j = 0; j < c2; j++)
- {

- result[i][j] = 0;
-
- for (k = 0; k < c1; k++)
- {
- result[i][j] += A[i][k] * B[k][j];
- }
- }
- }
• }
-
- /* Function to compare two matrices */
- int compareMatrix(int **A, int **B, int rows, int cols)
- {
- int i, j;
-
- for (i = 0; i < rows; i++)
- {
- for (j = 0; j < cols; j++)
- {
- if (A[i][j] != B[i][j])
- return 0;
- }
- }
-
- return 1;
- }
-
- /* Function to allocate a matrix dynamically */
- int **allocateMatrix(int rows, int cols)
- {
- int **matrix;
- int i;
-
- matrix = (int **)malloc(rows * sizeof(int *));
-
- for (i = 0; i < rows; i++)
- {
- matrix[i] = (int *)malloc(cols * sizeof(int));
- }

-
- return matrix;
- }
-
- /* Function to free matrix memory */
- void freeMatrix(int **matrix, int rows)
- {
- int i;
-
- for (i = 0; i < rows; i++)
- {
- free(matrix[i]);
- }
-
- free(matrix);
- }
-
- /* Operation 1: Matrix Addition Associativity */
- void additionAssociativity()
- {
- int m, n;
- int **A, **B, **C;
- int **AB, **BC;
- int **left, **right;
-
- printf("\nEnter number of rows (m): ");
- scanf("%d", &m);
-
- printf("Enter number of columns (n): ");
- scanf("%d", &n);
-
- A = allocateMatrix(m, n);
- B = allocateMatrix(m, n);
- C = allocateMatrix(m, n);
-
- AB = allocateMatrix(m, n);
- BC = allocateMatrix(m, n);
- left = allocateMatrix(m, n);
- right = allocateMatrix(m, n);

-
- `printf("\nEnter Matrix A:\n");`
- `readMatrix(A, m, n);`
-
- `printf("\nEnter Matrix B:\n");`
- `readMatrix(B, m, n);`
-
- `printf("\nEnter Matrix C:\n");`
- `readMatrix(C, m, n);`
-
- `/* (A + B) + C */`
- `addMatrix(A, B, AB, m, n);`
- `addMatrix(AB, C, left, m, n);`
-
- `/* A + (B + C) */`
- `addMatrix(B, C, BC, m, n);`
- `addMatrix(A, BC, right, m, n);`
-
- `printf("\n(A + B) + C =\n");`
- `displayMatrix(left, m, n);`
-
- `printf("\nA + (B + C) =\n");`
- `displayMatrix(right, m, n);`
-
- `if (compareMatrix(left, right, m, n))`
- `printf("\nAssociative property of matrix addition is VERIFIED.\n");`
- `else`
- `printf("\nAssociative property of matrix addition is NOT VERIFIED.\n");`
-
- `freeMatrix(A, m);`
- `freeMatrix(B, m);`
- `freeMatrix(C, m);`
- `freeMatrix(AB, m);`
- `freeMatrix(BC, m);`
- `freeMatrix(left, m);`
- `freeMatrix(right, m);`
- `}`
-
- `/* Operation 2: Matrix Multiplication Associativity */`

- void multiplicationAssociativity()
- {
- int m, n, p, q;
- int **A, **B, **C;
- int **AB, **BC;
- int **left, **right;
-
- printf("\nEnter m: ");
- scanf("%d", &m);
-
- printf("Enter n: ");
- scanf("%d", &n);
-
- printf("Enter p: ");
- scanf("%d", &p);
-
- printf("Enter q: ");
- scanf("%d", &q);
-
- A = allocateMatrix(m, n);
- B = allocateMatrix(n, p);
- C = allocateMatrix(p, q);
-
- AB = allocateMatrix(m, p);
- BC = allocateMatrix(n, q);
-
- left = allocateMatrix(m, q);
- right = allocateMatrix(m, q);
-
- printf("\nEnter Matrix A (%d x %d):\n", m, n);
- readMatrix(A, m, n);
-
- printf("\nEnter Matrix B (%d x %d):\n", n, p);
- readMatrix(B, n, p);
-
- printf("\nEnter Matrix C (%d x %d):\n", p, q);
- readMatrix(C, p, q);
-
- /* (A × B) × C */

- multiplyMatrix(A, B, AB, m, n, p);
- multiplyMatrix(AB, C, left, m, p, q);
-
- /* $A \times (B \times C)$ */
- multiplyMatrix(B, C, BC, n, p, q);
- multiplyMatrix(A, BC, right, m, n, q);
-
- printf("\n(A x B) x C =\n");
- displayMatrix(left, m, q);
-
- printf("\nA x (B x C) =\n");
- displayMatrix(right, m, q);
-
- if (compareMatrix(left, right, m, q))
- printf("\nAssociative property of matrix multiplication is VERIFIED.\n");
- else
- printf("\nAssociative property of matrix multiplication is NOT VERIFIED.\n");
-
- freeMatrix(A, m);
- freeMatrix(B, n);
- freeMatrix(C, p);
- freeMatrix(AB, m);
- freeMatrix(BC, n);
- freeMatrix(left, m);
- freeMatrix(right, m);
- }
-
- /* Main function */
- int main()
- {
- int choice;
-
- do
- {
- printf("\n===== MATRIX OPERATIONS =====\n");
- printf("1. Matrix Addition Associativity\n");
- printf("2. Matrix Multiplication Associativity\n");
- printf("3. Exit\n");
- printf("Enter your choice: ");

```

•     scanf("%d", &choice);
•
•     switch (choice)
•     {
•         case 1:
•             additionAssociativity();
•             break;
•
•         case 2:
•             multiplicationAssociativity();
•             break;
•
•         case 3:
•             printf("\nProgram exited successfully.\n");
•             break;
•
•         default:
•             printf("\nInvalid choice! Please try again.\n");
•     }
•
•     } while (choice != 3);
•
•     return 0;
• }

```

Test Case Table:

Test Case	Choice	Input	Expected Output
1	1	A= 1 2 3 4 , B = 5 6 7 8, C= 9 10 11 12	(A+B) + C = 15 18 21 24 A + (B+C) = 15 18 21 24
2	2	A = 1 2 3 4, B= 5 6 7 8, C = 1 0 0 1	(A * B)*C = 19 22 43 50 A * (B*C)= 19 22 43 50
3	3	Exit	Program exited successfully
4	4	Any invalid menu choice	Invalid choice

Compilation and Execution:

The program has been compiled and executed successfully.

Test Case 1:

```
Enter your choice: 1

Enter number of rows (m): 2
Enter number of columns (n): 2

Enter Matrix A:
Enter elements of the matrix:
1 2 3 4

Enter Matrix B:
Enter elements of the matrix:
5 6 7 8

Enter Matrix C:
Enter elements of the matrix:
9 10 11 12

(A + B) + C =
15      18
21      24

A + (B + C) =
15      18
21      24

Associative property of matrix addition is VERIFIED.
```

Test Case 2:

```
Enter your choice: 2

Enter m: 2
Enter n: 2
Enter p: 2
Enter q: 2

Enter Matrix A (2 x 2):
Enter elements of the matrix:
1 2 3 4

Enter Matrix B (2 x 2):
Enter elements of the matrix:
5 6 7 8

Enter Matrix C (2 x 2):
Enter elements of the matrix:
1 0 0 1

(A x B) x C =
19      22
43      50

A x (B x C) =
19      22
43      50

Associative property of matrix multiplication is VERIFIED.
```

Test Case 3:

```
===== MATRIX OPERATIONS =====
1. Matrix Addition Associativity
2. Matrix Multiplication Associativity
3. Exit
Enter your choice: 3

Program exited successfully.
```

Test Case 4:

```
(kali㉿kali)-[~]  
$ ./a.out
```

```
===== MATRIX OPERATIONS =====  
1. Matrix Addition Associativity  
2. Matrix Multiplication Associativity  
3. Exit  
Enter your choice: 4  
  
Invalid choice! Please try again.
```